

Understanding Python Metaprogramming

Ensuring Dynamic Code Execution and Extensibility

Python classes and objects are runtime values

- Classes are first-class objects in the Python runtime, meaning they can be passed to functions, stored in variables, or created dynamically.
- Instance introspection: an object retains a direct link to its defining class through the special '`__class__`' attribute.
- Class metadata: classes expose their parent structure via '`__bases__`' and their full inheritance hierarchy via '`__mro__`' (Method Resolution Order).
- This runtime visibility forms the foundation of Python metaprogramming.



01_runtime_object_model.py

```
class Account:
    pass

account = Account()

print(account.__class__)
# Outputs: <class '__main__.Account'>

print(Account.__name__)
# Outputs: 'Account'

print(Account.__bases__)
# Outputs: (<class 'object'>,)

print(Account.__mro__)
# Outputs: MRO tuple hierarchy...
```

Why It Matters & Where This Is Used

- Every other technique in these notes depends on this: before code can validate, register, or generate a class, it first has to ask questions about that class at runtime.
- Introspection lets a function react to what an object's type actually is, without hardcoding a class name anywhere in the function itself.
- Where this is used: serializers use `__class__` to decide how to represent an object; test frameworks walk `__mro__` to implement isinstance-style checks; ORMs inspect `__bases__` to find a model's parent table.



01_runtime_object_model.py

```
class Account:
    pass

account = Account()

# __class__ links an instance back to its class
print(account.__class__)
# Outputs: <class '__main__.Account'>

# __name__ is the class's own name, as text
print(Account.__name__)
# Outputs: 'Account'

# __bases__ lists the classes it inherits from
print(Account.__bases__)
# Outputs: (<class 'object'>,)

# __mro__ is the FULL lookup order Python will search
print(Account.__mro__)
# Outputs: MRO tuple hierarchy...
```

Intercepting attribute access with `__getattr__`

- The '`__getattr__`' magic method is a fallback mechanism. It executes ONLY when the requested attribute cannot be found through normal lookup.
- Useful for computed properties, proxy patterns, adapters, and resolving fallback configurations dynamically.
- Safety note: `self.values` here is a plain dictionary, so `self.values[name]` simply looks up the missing attribute's name as a dictionary key.
- Always raise a standard 'AttributeError' if the requested key cannot be resolved.

```
02_getattr_dynamic_attributes.py

class Settings:
    def __init__(self, values):
        self.values = values  # a dict, e.g. {"debug": True}

    def __getattr__(self, name):
        # name is the missing attribute, e.g. "debug"
        # self.values[name] looks it up as a dict key
        try:
            return self.values[name]
        except KeyError:
            raise AttributeError(name)

settings = Settings({"debug": True})
print(settings.debug)  # Outputs: True
print(settings.host)   # AttributeError!
```

Why It Matters & Where This Is Used

- Lets an object built around a flexible dict or JSON source still be read with clean dot syntax (settings.debug), without pre-declaring every possible key as a class attribute.
- `__getattr__` only fires on a FAILED lookup, so real attributes like `self.values` are completely unaffected and never cause recursion.
- Where this is used: configuration wrappers, proxy or adapter objects, and lazily computed attributes.



02_getattr_dynamic_attributes.py

```
class Settings:
    def __init__(self, values):
        # A real attribute, found by normal lookup, so
        # __getattr__ is never triggered for this line
        self.values = values

    def __getattr__(self, name):
        # self.values is a dict; name is the missing
        # attribute's name, used here as the dict key
        try:
            return self.values[name]
        except KeyError:
            raise AttributeError(name)

settings = Settings({"debug": True})
print(settings.debug) # Outputs: True
print(settings.host) # AttributeError!
```

Customizing attribute assignment with `__setattr__`

- The '`__setattr__`' method intercepts ALL attribute assignments (e.g. `obj.name = value`), providing validation and normalization gates.
- Preventing recursion: `self.name = value` is itself an assignment, so writing it inside '`__setattr__`' routes straight back into this same method, which calls itself again, forever. '`object.__setattr__`' bypasses the override to perform the real write.
- Ensuring predictable APIs: dynamic changes should validate type constraints without altering intuitive variable binding behaviour.
- Errors should fail early with helpful `TypeError` or `ValueError` summaries.



03_setattr_namespace_control.py

```
class Positive:
    def __setattr__(self, name, value):
        if name == "age" and value < 0:
            raise ValueError("age >= 0 required")
        # Bypass custom write to prevent loops
        object.__setattr__(self, name, value)

p = Positive()
p.age = 30 # Success
p.age = -5 # ValueError!
```

Why It Matters & Where This Is Used

- `__setattr__` intercepts EVERY assignment, so it is the natural place for validation that must run no matter how or when a value gets set.
- `object.__setattr__` performs the real write using Python's default assignment logic instead of this class's own override, which is exactly what avoids calling this same method again forever.
- Where this is used: ORMs mark a field 'dirty' the moment it changes; frozen or read-only objects; keeping a cache or UI widget automatically in sync with a changing field.



03_setattr_namespace_control.py

```
class Positive:
    def __setattr__(self, name, value):
        # Runs for EVERY assignment, e.g. p.age = 30
        if name == "age" and value < 0:
            raise ValueError("age >= 0 required")
        # self.name = value here would call THIS SAME
        # method again, forever. object.__setattr__
        # bypasses the override to avoid that loop.
        object.__setattr__(self, name, value)
```

```
p = Positive()
p.age = 30 # Success
p.age = -5 # ValueError!
```

Constructing classes dynamically with type()

- The 'type' function has two roles: type(obj) inspects an object's type, while type(name, bases, dict) constructs a new class.
- The bases argument is a tuple of parent classes. An empty tuple, (), means the class inherits only from Python's default base, object, the same as writing 'class Person:' with no explicit parent.
- The namespace argument is a dictionary containing the class's attributes, methods, and closures.
- Prefer standard, static class definitions when the structure is known ahead of time.

```
04_dynamic_class_construction_type.py

def greet(self):
    return f"Hello, I am {self.name}"

# Dynamic class construction
Person = type(
    "Person",           # class name
    (),                 # bases: empty = inherits object
    {"name": "Ada", "greet": greet}, # namespace: attrs & methods
)

p = Person()
print(p.greet())      # 'Hello, I am Ada'
print(Person.__name__) # 'Person'
```


Why It Matters & Where This Is Used

- Lets code build a class whose shape (its name, attributes, and methods) is only known while the program is RUNNING, not while the source code is being written.
- The 3-argument form of `type()` is exactly what the ``class`` keyword compiles down to internally: name, bases (parents), and a namespace dict of attributes and methods.
- Where this is used: ORMs generating a model class from a database table's columns; `dataclasses.make_dataclass`; turning a JSON Schema or plugin manifest into real classes.



04_dynamic_class_construction_type.py

```
def greet(self):
    return f"Hello, I am {self.name}"

# Same 3 things the `class` statement builds:
# name, bases (parents), namespace (body)
Person = type(
    "Person",
    (),
    {"name": "Ada", "greet": greet},
)

p = Person()
print(p.greet())      # 'Hello, I am Ada'
print(Person.__name__) # 'Person'
```

Binding methods dynamically at runtime

- Functions are first-class callable objects and can be attached to classes or instances as attributes dynamically at runtime.
- Class-level binding: assigning a function to a class instantly exposes it as a standard, bound instance method to all active instances.
- Useful for extensible plugin architectures, lazy-loaded utilities, and programmatically generated database models.
- Always maintain a clear naming convention and testing strategy for dynamically bound methods.



05_dynamic_method_binding.py

```
class Report:
    pass

def summary(self):
    return "report ready"

# Bind function as a class method
Report.summary = summary

r = Report()
print(r.summary()) # 'report ready'
```

Why It Matters & Where This Is Used

- Adds behaviour to a class after the fact, even from code living in a completely different file, without ever editing the class's own original definition.
- Because functions are ordinary first-class objects, assigning one to a class attribute makes every instance, even ones created earlier, able to call it as a normal method.
- Where this is used: plugin and mixin systems attaching handler methods to a shared base class; API client libraries generating one method per endpoint; monkey-patching a method inside a test.



05_dynamic_method_binding.py

```
class Report:
    pass

def summary(self):
    return "report ready"

# Attaching a plain function to a class that already
# exists makes it behave like an ordinary method
Report.summary = summary

r = Report()
print(r.summary()) # 'report ready'
```

Coordinating class creation with Metaclasses

- A metaclass is a class of a class, defining how classes themselves are constructed and instantiated by the Python compiler.
- Subclass 'type' to declare a custom metaclass. Override '__new__' to intercept and modify the class namespace before it is created.
- Ideal for enforcing design constraints at import time, preventing incorrect configurations before classes are ever instantiated.
- Always keep metaclass logic small, isolated, and documented to avoid adding unnecessary structural complexity.

```
06_metaclass_interface_verifier.py

class RequireRun(type):
    def __new__(mcls, name, bases, namespace):
        if name != "Base" and "run" not in namespace:
            raise TypeError(f"{name} must define run()")
        return super().__new__(mcls, name, bases, namespace)

class Base(metaclass=RequireRun):
    pass

class Job(Base):
    def run(self): return "ok"

# class BrokenJob(Base): pass # TypeError!
```

Why It Matters & Where This Is Used

- Enforces a contract at IMPORT time, the moment Python reads the class definition, instead of waiting until someone calls a missing method and gets a plain `AttributeError` far from the real mistake.
- `__new__` runs BEFORE the class object exists, which is the one thing only a metaclass can do.
- Where this is used: Django's model metaclass processes fields and Meta options at class-creation time; SQLAlchemy's declarative base works the same way; Python's own `abc.ABCMeta` and `enum.EnumMeta` are both built as metaclasses internally.



06_metaclass_interface_verifier.py

```
class RequireRun(type):
    def __new__(mcls, name, bases, namespace):
        # Runs before the class object even exists
        if name != "Base" and "run" not in namespace:
            raise TypeError(f"{name} must define run()")
        return super().__new__(
            mcls, name, bases, namespace)

class Base(metaclass=RequireRun):
    pass

class Job(Base):
    def run(self): return "ok"

# class BrokenJob(Base): pass # TypeError!
```

Using Metaclasses Sparingly

- Rule of Least Power: always use the simplest mechanism that cleanly solves the problem at hand.
- Prefer an ordinary class, a simple class decorator, or a plain validation function first.
- Reach for a custom metaclass only when the class creation process itself must be controlled, such as modifying the namespace before the class exists.
- Keep metaclass logic small, isolated, and documented, since two libraries each supplying their own metaclass for the same class can conflict with one another.



07_metaclass_design_principles.py

```
class MetaBase(type):
    def __new__(mcls, name, bases, d):
        # Modifies raw dict before class exists
        d["meta_tag"] = True
        return type.__new__(mcls, name, bases, d)

class Widget(metaclass=MetaBase):
    pass

print(Widget.meta_tag)  # True
```

Why It Matters & Where This Is Used

- Rule of Least Power in practice: metaclasses are powerful, but that power comes at the cost of readability, so they should stay a last resort.
- A metaclass can inspect and rewrite the namespace dict before the class object exists at all, a capability no simpler mechanism provides.
- Where this is used: Django's model metaclass, SQLAlchemy's declarative base, and Python's own `abc.ABCMeta` and `enum.EnumMeta` all rely on tightly scoped metaclasses that each do one job well.



07_metaclass_design_principles.py

```
class MetaBase(type):
    def __new__(mcls, name, bases, d):
        # `d` is still a plain dict here, not a class yet
        d["meta_tag"] = True
        return type.__new__(mcls, name, bases, d)

class Widget(metaclass=MetaBase):
    pass

print(Widget.meta_tag)  # True
```

6 Activities to Consolidate Concepts

1

Dynamic Metadata Introspection

Read and summarize dunder attributes.

2

Dynamic Configuration Wrapper

Build a dot-notation dictionary adapter using `__getattr__`.

3

Strict Attribute Validation

Intercept assignments with `__setattr__` to enforce schemas.

4

Dynamic Class Factory

Construct customizable classes on the fly using `type()`.

5

Dynamic API Method Generator

Bind helper methods programmatically based on string endpoints.

6

Metaclass Interface Verifier

Enforce class validations and docstring presence at compile time.